

LSML 5

Лин регрессия

Hashing trick

Вспомним про Broadcast еще раз

$X \rightarrow \text{read} \rightarrow \text{RDD} \rightarrow B_1, \dots, B_n$

`sc.map(f-compute)`

`w = np.rand(d)` – генерим стартовые веса одним из методов

`w_b = sc.broadcast(w)` – метод для раскидывания вектора весов между всеми воркерами

Преимущество – вектор весов в полном объеме раскидывается на воркеры перед основными вычислениями

Вспомним про Accumulator еще раз

Обратный подход. В аккумулятор мы будем складывать статистики со всех воркеров

```
num_errors = 0  
ac_n_e = sc.accumulator(num_errors)
```

В воркере по ходу обработки данных: `ac_n_e.add(1)` – счетчик учета «проблемных» логов

Лин регрессия

LR with MSE -> какое решение этой задачи нам известно?

Лин регрессия

LR with MSE -> какое решение этой задачи нам известно? Да, это градиентный спуск

Давайте перейдем к его формуле:

$$w_{t+1} = w_t - \lambda * \nabla_w \mathcal{L}(X)$$

X = Пб (много данных) $f_1, f_2, f_3, \dots, f_d$

$\nabla_w \mathcal{L}(X) = \sum_{i=1}^N \nabla_w l(x_i, y) \rightarrow$ сумма ошибок от каждого экземпляра данных, и эти вычисления мы можем распараллелить

$$\nabla_w l \rightarrow \text{sc.map}(\nabla_w l). \text{sum}()$$

Лин регрессия

```
sc = read_file(path).cache()
```

```
w = np.rand(d)
```

```
w_b = sc.broadcast(w)
```

```
for i in epochs:
```

```
     $\nabla_w L = \text{sc.map}(\nabla_w l). \text{sum}()$ 
```

```
     $w = w - \lambda * \nabla_w L$ 
```

```
    w_b = sc.broadcast(w)
```

Hashing trick

$X \in \mathbb{R}^{n \times d}$, $d \ll \infty$, изначально размер-ть пр-ва не такая существенная, но рассмотрим кейс когда d существенно большое

$h : X \rightarrow [0, \dots, m]$, хотим перейти к более сжатому пр – ву признаков, и используем хэш ф.

$h(x) = h(x)$ – детерминированная

Работаем с примером – данные для рекомендательной системы.

Что-то кодируем (one, etc.), какие-то фичи просто нормализуем.

По одному юзеру после всех преобразований получаем оч большое пр-во фичей.

Важно, оно выходит разреженным. Попытаемся его ужать.

Hashing trick

$X \rightarrow X'$

$d \gg m$

$X_i = x_1, x_2, x_3, x_4, \dots, x_d$ - полный вектор значений/фичей у объекта

$h(x_i) = h_i$ — индекс подставляемого значения $\rightarrow 0, 0, 0, \dots, x_i$ (на h_i позиции), $0, 0$ – вектор размерности m

Основная идея подхода: мы считаем хэш от каждого вектора изначальных признаков и отображаем их в вектор с номером $i \rightarrow [0, m]$, тем самым мы получаем «объединение» нескольких признаков в одной колонке.

Пример: $h(feature_1) = h(feature_2) = h(feature_3) \rightarrow 3$ признака из изначального пр-ва признаков попадают в одну общую колонку в нашем пр-ве X' . У каждого экземпляра данных будет ненулевое значение в этой новой колонке или от или от $feature_1$, или от $feature_2$, или от $feature_3$. Если ненулевых значений несколько, то решаем задачу коллизии.

Данный подход похож чем-то на процесс регуляризации модели, поэтому помогает ей также успешно обучиться не попав в состояние оверфита (когда у нас обычно фичей очень много)

Hashing trick

$X \rightarrow X'$

$d \gg m$

$X_i = x_1, x_2, x_3, x_4, \dots, x_d$ - полный вектор значений/фичей у объекта

$h(x_i) = h_i$ — индекс подставляемого значения $\rightarrow 0, 0, 0, \dots, x_i$ (на h_i позиции), $0, 0$ — вектор размерности m

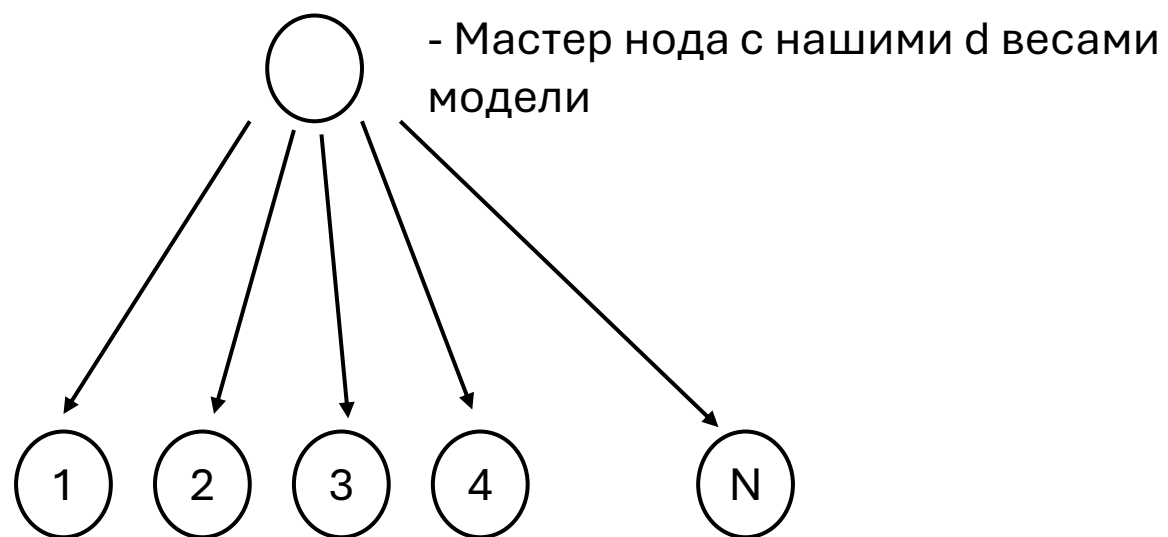
Основная проблема – получение коллизий в работе ф-ии

Как их можно решить:

1. Выбор случайного из значений, попадающих на индекс
2. Выбор удобной арифметич. ф-ии: avg, min, max

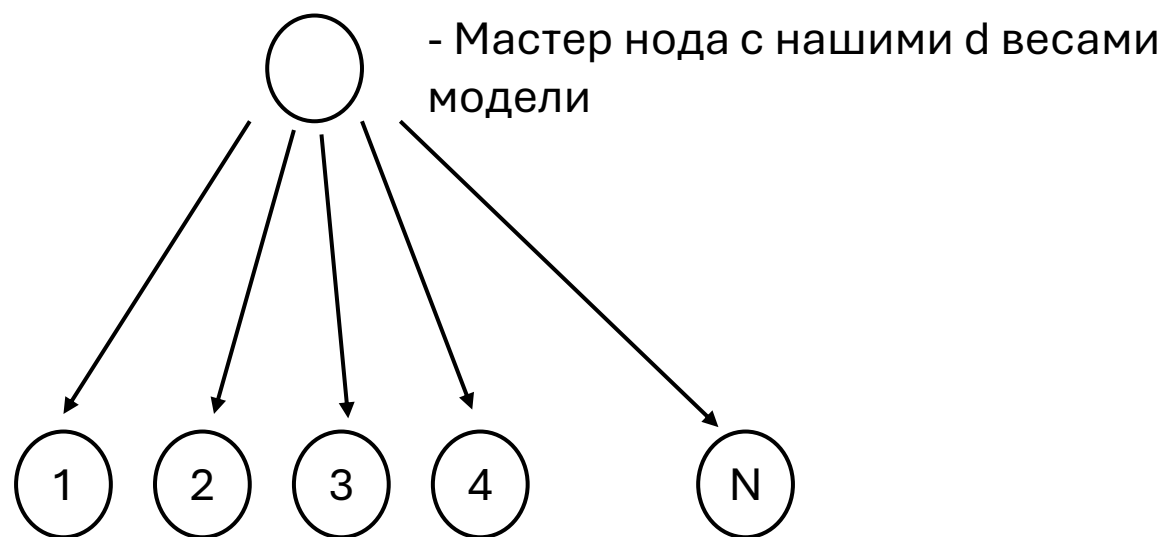
Данная концепция хэширования фичей успешно работает под капотом Vopal Wabbit, работу которого мы рассмотрим на семинаре

Заметки по системе передачи данных между машинами кластера (например весов модели)



За какую асимптотику мы сможем передать все веса на наших машины воркеры?

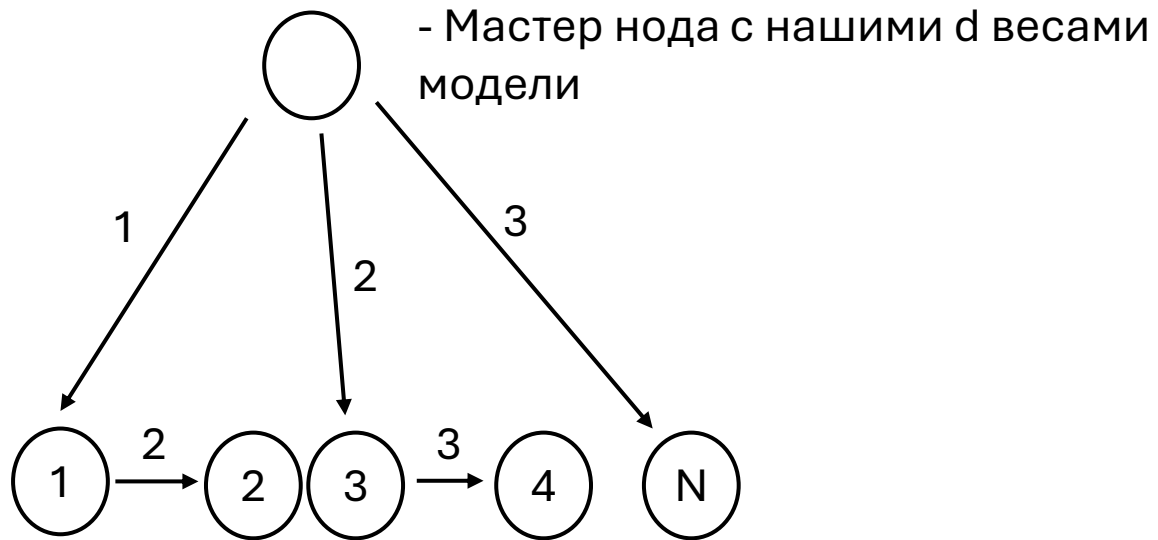
Заметки по системе передачи данных между машинами кластера (например весов модели)



За какую асимптотику мы сможем передать все веса на наши машины воркеры?

По идее за $O(N)$. Как можно быстрее?

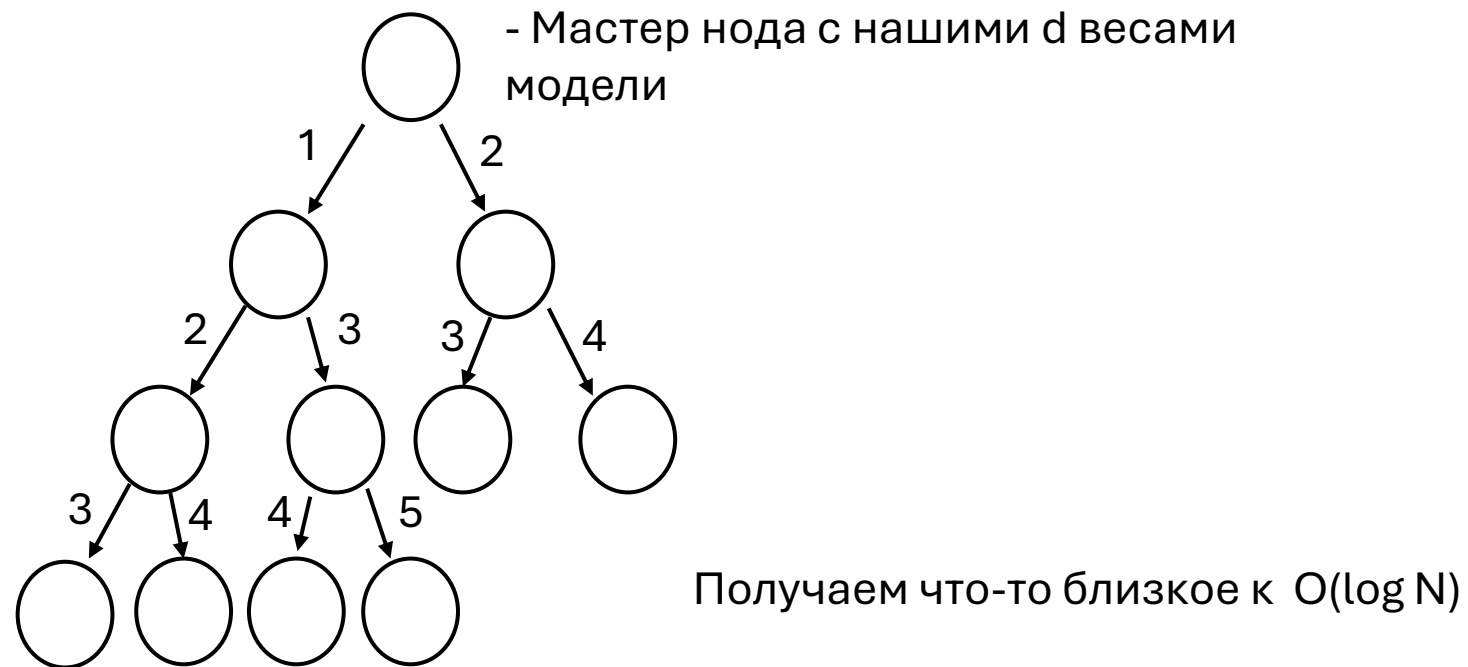
Заметки по системе передачи данных между машинами кластера (например весов модели)



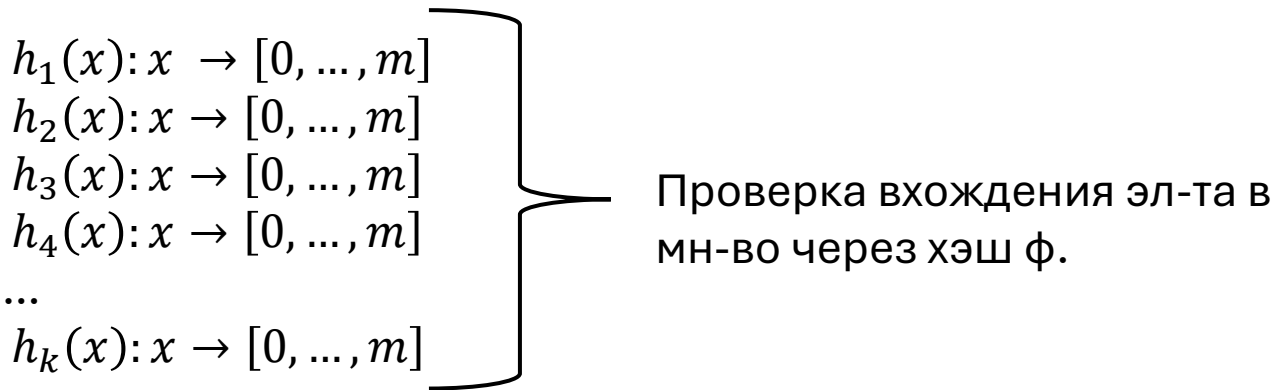
За какую асимптотику мы сможем передать все веса на наших машины воркеры?

По идее за $O(N)$. Как можно быстрее?

Заметки по системе передачи данных между
машинами кластера (например весов модели)



Блум фильтр



0	0	0
1 -> $h_3(x_1)$	1 -> $h_1(x_2)$	1
0	0	0
0	1 -> $h_2(x_2)$	1
1 -> $h_1(x_1)$	0	1
1 -> $h_2(x_1)$	0	1
0	1 -> $h_3(x_2)$	1

Битовая маска с записью результатов хэшей

Если хотя бы одно значение $h_i(x_3) = 0$ – точно не в множестве

В других случаях – возможно эл-т есть в мн-ве, с вер-тью ошибки p

Будет ли новый эл-т x_3 в множестве?

Блум фильтр

Вер-ть $h_i = 1 \rightarrow \frac{1}{m}$ (учитываем равномерность ф-ий)

$$h_i = 0 \rightarrow 1 - \frac{1}{m}$$

$$P(\text{от всех } h \text{ ф-ий значение } 0) \rightarrow \left(1 - \frac{1}{m}\right)^k$$

$$P(\text{на позиции } i \text{ будет } 0 \text{ от хэш ф. от всех эл-ов}) \rightarrow \left(1 - \frac{1}{m}\right)^{kn}$$

$$P(\text{на позиции } i \text{ будет } 1 \text{ от всех ф-ий по всем эл-ам}) \rightarrow \left(\left(1 - \frac{1}{m}\right)^{kn}\right)^k$$

Выведем приближ и более простую зависимость:

$(1 - e^{-\frac{kn}{m}})$ – вер-ть, что все биты в нашей маске будут 1 к моменту добавления нового эл-та из последовательности.

Блум фильтр

Тогда получаем следующие идеал. значения:

p – вер-ть ошибки вхождения эл-ва в мн-во для размера n

$$m = -\frac{n \ln(p)}{(\ln 2)^2} \quad k = -\frac{\ln(p)}{\ln(2)}$$

Тогда при $n = 10^6$ и $p = 0.01$ получаем $m = 10^7$ (бит) и $k = 7$

Доп инф к примеру: работаем с проверкой ссылок и блокировкой из них «опасных»

Пусть каждая ссылка будет строкой до 1024 байт

Тогда: объем памяти на хранение всех n логов и поиска по ним $\rightarrow 10^6 * 1024$ байт = 1 Гб данных

Хранение битовой маски по этому списку логов и проверка вхождения новых через 7 хэш ф-ий $\rightarrow 10^7$ бит = 1,2 Мб

Другая сторона использования: удобная фильтрация ссылок по маске до этапа join данных, чтоб не шаффлировать между воркерами ненужные данные